WFC 오픈소스를 활용하여 자동 던전 생성 알고리즘 구현하기

202555302 강백호

- 고른 이유 : 게임 개발에 관심이 많아서 게임과 관련있는 오픈소스는 어떤게 있을지 확인해보던 중 WFC(파동 함수 붕괴) 오픈소스를 활용하여 특정 제약을 만족하는 무작위 2차원 배열구조를 만들 수 있다는 사실을 깨달았습니다. 이 알고리즘을 활용하면 로그라이크(무작위성 요소를 특징으로 하는 반복적인 플레이를 요하는 게임장르) 장르의 무작위 던전을 생성하거나, 무작위 지형을 생성하는 것 등에 적용할 수 있을 것 같아 흥미가 생겨 WFC 오픈소스를 선택했습니다.

- 원리 설명

우선 기존 WFC 알고리즘에 대해 설명드리겠습니다.

WFC 알고리즘은 Model.cs의 Model 추상 클래스에 구현되어 있습니다.

Model 클래스는 생성할 2차원 배열을 가지며 각 배열의 칸에 들어갈 수 있는 여러 후보(tile)가 있습니다.

여기서 propagator라는 후보들끼리 특정 방향에 대해 인접할 수 있는지 그 규칙을 담아둔 배열이 있는데 이는 Model 클래스를 상속한 자식 클래스에서 구현합니다.

1. Model은 프로그램 실행전에 clear 함수에서 던전 생성 시작전 준비를 하는데 이 과정에서 propagator가 적용되어 불가능한 후보들을 미리 제외시켜둡니다.
2. 프로그램이 실행되면 확정되지않은 각 칸에는 여러 후보 타일이 중첩된 상태로 존재합니다.
3. Model은 Heuristic enum에 따라 어떤 칸을 선택할지 결정하는데 보통은 엔트로피(불확실도)가 낮은, 즉 후보군이 적은 칸을 우선적으로 탐색합니다.

4. 어떤 칸을 탐색할지 정해지면 그 칸을 관측(후보 타일 중 하나를 확정하는 것)하는데 후보 타일마다 가중치가 다를 수 있습니다. 그러한 가중치를 적용해서 무작위 타일 하나를 뽑습니다.
5. 특정 칸에 특정 타일이 확정되는 사건은 확정된 칸과 인접한 칸에도 영향을 주게 됩니다. 왜냐하면 확정된 타일에 의해 인접한 칸에 올 수 없는 타일이 있을 수 있기 때문에 어떤 칸에 타일이 확정되고 나면 그와 인접한 칸에 전파(소식을 알리는 것)시켜주어야합니다. Propagate 함수를 호출하여 주변타일에 전파시켜주면 주변 타일에 후보로 있던 타일 중 확정된 타일과 만날 수 없는 타일들을 후보들을 지우게(Ban 함수 호출) 됩니다.
6. 3~5 과정을 반복하게 되면 특정 제약을 만족하는 무작위 2차원 배열구조를 생성할 수 있습니다.

이제 기존 WFC 알고리즘을 기반으로 한 무작위 던전 생성 알고리즘에 대해 설명리겠습니다.

스크립트는 DW(DungeonWave.cs)와 DP(DungeonPrototype.cs)가 있습니다. DW는 던전 생성 알고리즘을 구현해놓은 것이고 DP에서 이렇게 구현된 함수를 호출하여게임을 진행합니다. 먼저 DW를 살펴보면 기존 Model클래스와 크게 다르지 않습니다. 차이점 위주로 짚어드리겠습니다.

타일간의 규칙을 정의하는 propagator가 nBT(neighborsByTile)의 변수로 DW내에서 구현되어있다는 것입니다. 어떻게 규칙이 정의되는지를 간단하게 설명드리겠습니다. 이번 프로젝트에서는 한 칸에 올 수 있는 타일을 16종으로 설정했습니다. 중심에 방이 있고 북동남서 방향으로 길이 존재할 수 있으므로 2^4=16개의 가능성을 가지기 때문입니다. 북동남서는 각각 1,2,4,8의 이진수 값을 가지고 이를 통해 타일의 종류를 확인할 수 있습니다. 예를들어 북쪽과 남쪽길이 있는 타일은 북(1) + 남(4) = 5로 표현되는 것이죠. 이렇게 타일의 종류를 이진수연산으로 나타내어 생성자에서 타일들끼리 특정 방향에서 인접할 수 있는지에 대한 정보를 nBT 변수에 저장해두면 propagator 규칙을 간단하게 정의할 수 있습니다. 이렇게 정의된 규칙을 가지고 기존 WFC의 알고리즘과 같이 칸을 하나 선택하여 관측하고 이를 인접한 칸에 전파하면서 상태를 갱신하는 과정을 반복하면 던전 생성 알고리즘을 DW에 구축할 수 있습니다.

하지만 여기까지만보면 Model 추상 클래스를 상속받지 않고 따로 DW를 만드는 이유에 대해 의문이 생길 수 있습니다. 핵심은 실행 방식에 차이가 있습니다. 이번 프로젝트의 목적은 단순히 던전을 생성하는 것이 아닌 생성과정을 한 단계씩 보여주는데에 있기 때문에 기존 Model 클래스를 그대로 사용하게 되면 생성 과정을 단계로 나누어 보여줄 수 없습니다. 따라서 저는 알고리즘만 DW에 구현해두고 구체적인 실행은 DP에서 담당하도록 했습니다.

그럼 이제 DP의 원리에 대해 말씀드리겠습니다.

DP는 DW에서 제공한 던전 생성 및 경로 제공 알고리즘을 사용하여 게임을 실행하고 진행하는 게임 진행 전반을 담당합니다. 프로젝트의 내용이 비교적 간단하므로 게임의 진행 상황은 Phase enum으로 관리됩니다. 코드가 길지만 게임 진행에 핵심적인 부분 위주로 살펴보겠습니다. update함수에서 매 프레임마다 phase에 따라 필요한 동작을 수행하는데 Advance 함수에서 Phase에 따라 propagate(),observe() 함수를 호출하여 던전을 실질적으로 생성합니다. 그리고 단계별로 설명 텍스트를 띄우거나 칸에 색을 입히는 등의 동작은 Render 함수에서 담당합니다.

이렇게 기존 WFC와 달리 이번 프로젝트의 목적에 맞게 WFC가 던전을 생성하는 과정을 순차적으로 보여주고 설명하기 위해 무작위 2차원 배열을 생성하는 알고리즘을 구현해둔 부분과 이를 순차적으로 실행하며 관리하는 부분을 분리했습니다.

- 내가 바꾼 것

기존 WFC 오픈소스는 타일에 사용할 이미지를 입력받아 이를 조합하여 이미지 파일을 반환하는 기능을 합니다. 제 프로젝트는 unity 게임엔진에서 던전을 생성하고 플레이어가 시작지점에서 출구지점까지 이동하는 간단한 체험형 게임이므로 핵심 알고리즘 부분을 DW와 DP로 분리하여 던전 생성에 맞게 수정하였고 플레이어 입력을 받아 각 타일의 가중치를 설정할 수 있도록 하였고 생성모드도 두가지로 구분하였습니다. Ui도 배치하여 정보가 적절하게 제공될 수 있도록 했습니다.

- 막힌 것, 해결방법

1. 우선 기존 오픈소스에서 던전 생성 알고리즘을 만들어야겠다는 기획은 구상했으나 이를 어떻게 구현할지 막막했습니다. 특히 제공된 오픈소스 코드를 어떻게 수정할지에 대해서도 고민이 많았습니다. 코드를 ai에게 분석시키고 설명해달라고 해봤지만 추상화된 설명만으로는 이해가 되지 않고 어떻게 코드를 수정해야할지에 대한 확신이 서지 않았습니다. 그래서 그냥 Model.cs, SimpleTiledModel.cs(Model 추상클래스를 상속받은 타일 생성 클래스, 던전 생성 알고리즘과 유사한 기능을 수행함) 코드를 한줄씩 모두 읽어봤습니다. 자주 사용되거나 중요한 역할을 하는 변수에 주목하여 역할을 파악하고 함수 단위의 기능을 제 나름대로 정리해보면서 대략적인 알고리즘 흐름을 파악했습니다. 코드 리뷰를 해보고나니 어떻게 저만의 방식으로 옮겨야할지에 대한 그림이 희미하게나마 그려졌던 것 같습니다.

2. 코드를 분석한 것을 바탕으로 던전을 생성하는 알고리즘은 성공적으로 만들었지만 결과물과 설명이 난잡하고 처음보는 입장에서는 전혀 이해할 수 없을 것처럼 보였습니다. AI를 활용하여 코드를 짜다보니 UI나 제 기획이 닿지 않은 부분에는 AI의 주관이 담겼는데 그러한 곳에서 문제가 생겼습니다. 결과 코드도 리뷰해보면서 어떻게 바꿔야할지 이것저것 고민해보는데도 정제되지 않은 듯한 느낌이 들어 세부적인 부분까지 처음부터 게임 진행 및 ui 부분에 대해 다시 검토했습니다. 빠르게 던전을 생성하는 모습을 보여주는 모드와 천천히 던전이 생성되는 과정을 설명과 함께 확인하면서 원리를 알아갈 수 있는 모드로 구분하고 던전 생성과정에서 나타나는 불필요한 정보는 제외하고 제공하는 정보도 플레이어의 입장에서 시각적으로 난잡하다고 느껴지지 않도록 최소화하고 배치도 적절하게 수정했습니다. 이렇게 수정하고 보니 기존 결과물에 비해 훨씬 보기좋고 직관적으로 화면을 구성할 수 있었습니다.

- AI 사용 내역과 내가 직접 수정한 부분

1. 구체적인 코드 구현은 Codex가 모든 부분을 담당했습니다.

2. 코드를 분석한 내용을 정리하는 과정에서 chat gpt로 재검토 했습니다.
3. Ui 배치 및 설명 텍스트 내용 게임 진행 방식은 제가 담당했습니다.
4. DW와 DP 코드에서 불필요하거나 단순화할 수 있는 부분은 Codex를 통해 수정했습니다.
5. Md 파일 작성에도 Codex를 활용했습니다

- 느낀점

사실 코딩을 하면서 깃허브에 올려져있는 오프소스 코드를 가공해서 저만의 방식으로 의미잇는 무언가를 만들어본 건 처음인데 굉장히 유익하고 의미있는 경험을 한 것 같습니다. 필요한 기능은 제가 직접 구현해보려는 강박이 있었는데 이번 프로젝트를 진행하면서 다른 사람들이 만들어놓은 코드를 분석해보고 사용해보는 것은 단순히 다른 사람의 코드를 베끼는 게 아니라 새로운 정보와 영감을 얻는 과정이라는 것을 느꼈습니다. 제가 직접 코드를 짜는 것도 재밌지만 다른 사람들은 어떤 알고리즘을 사용해서 어떻게 코드를 짰는지 코드 리뷰를 하는 것도 생각보다 재밌는 것 같습니다. 앞으로 자주 깃허브에 들어가봐야겠네요.